AI Product Development: iOS Course Description

Catalog Year: 2027, Required Hours: 720, Credits: 24

Foundation Core (Required Hours: 360, Credits: 12)	Credits	Hours
Introduction to AI Product Development	1.00	30.00
Product Thinking and Planning	2.00	60.00
Software Systems and Architecture	2.00	60.00
Software Development with AI	3.00	90.00
AI Agents and Automation	4.00	120.00

iOS Track (Required Hours: 360, Credits: 12)	Credits	Hours
Swift Fundamentals, AI-Accelerated	2.00	60.00
SwiftUI and App Architecture	3.00	90.00
Networking, Persistence and Cloud Backends	3.00	90.00
App Store, DevOps and App Marketing	2.00	60.00
Capstone and Career Launch	2.00	60.00

Foundation Core Courses (Required Hours: 360, Credits: 12)

Foundation Core (Required Hours: 360, Credits: 12)	Credits	Hours
Introduction to AI Product Development	**1.00**	**30.00**

The Introduction to AI Product Development course orients students to the world of AI-powered software development before any technical work begins. Students get a plain-language introduction to how AI systems work: what they are doing when they generate text, code, or decisions, and where they commonly fall short. The course also maps out what a modern technology company looks like: the roles on a product team and how those roles collaborate. The course closes with every student publishing a simple website built almost entirely with AI assistance.

Course Objectives:

- Explain in plain language how AI systems generate outputs and where they commonly fail.
- Identify the key roles on a software product team and describe how they collaborate.
- Describe how a technology organization is structured and where individual roles fit within it.
- Use AI tools to plan, build, and publish a simple website without prior technical experience.

Product Thinking and Planning	**2.00**	**60.00**

The Product Thinking and Planning course introduces the product development lifecycle from idea to launch: identifying a real user problem worth solving, gathering and prioritizing requirements, and translating a rough idea into a plan a team can act on. Students learn to write user stories and acceptance criteria and use a basic prioritization framework to decide what to build first and what to leave out.

Course Objectives:

- Describe the stages of a typical product development lifecycle from idea to launch.
- Identify a real user problem and articulate why it is worth solving.
- Write clear user stories and acceptance criteria.
- Gather, prioritize, and scope product requirements using a basic prioritization framework.
- Produce a one-page product brief that translates an idea into an actionable plan.

Software Systems and Architecture	**2.00**	**60.00**

The Software Systems and Architecture course builds the conceptual foundation every student needs before writing code. Students explore how computers process and store information, learn core data structures, develop a working understanding of how the internet functions, and study the design patterns and data modeling principles that shape how real software is architected. Rather than analyzing an arbitrary app, the course project has students take a product idea of their own and produce the architectural plan for it: the data model, the API surface, the client-server structure, and how information would need to move through the system end to end. The goal is to build the habit of designing before building. The course also introduces responsible data practices, asking students to weigh what their proposed product would collect, why it's needed, and what they're accountable for as its future developers.

Course Objectives:

- Demonstrate understanding of computer architecture, memory, storage, and operating system concepts.
- Apply common data structures to organize and manage information in a program.
- Explain how the internet works, including HTTP, DNS, and client-server communication, and how APIs move data between services.
- Apply software design patterns and data modeling principles to plan how a system should be structured.
- Produce an architectural plan for a product idea, including its data model, API surface, and system boundaries.
- Identify responsible data collection practices and describe the privacy implications of design decisions.

Software Development with AI	**3.00**	**90.00**

The Software Development with AI course is where students go from a plan to a working system. Students set up a professional development environment, write code in a general-purpose programming language, and use AI coding assistants and prompt engineering as core tools. Throughout the course, students critically evaluate the code AI produces, write tests to verify that AI-generated code actually does what it claims before it ships, and practice the judgment to know when to trust AI output and when to rewrite it. Ethical use is embedded in all projects through the principle that the developer is responsible for everything they ship, regardless of how it was written.

Course Objectives:

- Set up and navigate a professional software development environment including version control.
- Write and organize code using a general-purpose programming language.
- Use AI coding assistants and prompt engineering to accelerate development without sacrificing understanding.
- Critically evaluate AI-generated code for correctness, security, and appropriateness before shipping.
- Write tests to verify that AI-generated code behaves as intended before it ships.
- Build and deploy a scoped application that solves a defined, familiar problem using AI assistance.
- Design, build, and present a self-directed final project that turns a product idea into a working system.
- Articulate the developer's ethical responsibility for AI-assisted output.

AI Agents and Automation **4.00** **120.00**

The AI Agents and Automation course teaches students to design, build, and deploy agentic AI systems and automated workflows. Starting with no-code and low-code platforms, students progressively work with direct API integrations and agent frameworks to build systems that take real actions on behalf of users. The course covers how agents perceive inputs, reason through tasks, and produce outputs; how to design and sequence multi-step agentic workflows; and how to evaluate and debug agent behavior in production. Ethical considerations are central throughout: students examine the risks of autonomous systems, determine when human oversight is required, build appropriate guardrails, and reason through what accountability looks like when an agent makes a mistake that affects a real person.

Course Objectives:

- Build automated workflows using no-code and low-code platforms.
- Connect applications and services using APIs, webhooks, and event triggers.
- Explain how AI agents perceive inputs, reason through tasks, and produce outputs.
- Design and implement a multi-step agentic workflow using an agent framework.
- Evaluate and debug agent behavior using logging, testing, and prompt refinement.
- Identify failure modes of autonomous systems and implement appropriate guardrails.
- Apply human-in-the-loop design patterns in contexts where oversight is required.
- Articulate an accountability framework for agentic systems that affect real users.

iOS Track Courses (Required Hours: 360, Credits: 12)

iOS Track (Required Hours: 360, Credits: 12)	Credits	Hours

Swift Fundamentals, AI-Accelerated **2.00** **60.00**

The Swift Fundamentals, AI-Accelerated course teaches students to read and write idiomatic Swift, the language underneath every iOS app, with AI tools accelerating practice and review. The course closes the loop from language mechanics to working software: persisting simple data, pulling in packages, writing tests, and shipping a small command-line project.

Course Objectives:

- Write idiomatic Swift using optionals, collections, closures, and value vs. reference types.
- Model app state and data with enums, protocols, extensions, and generics.
- Handle failures explicitly with throws, do/catch, and the Result type.
- Encode and decode real-world JSON with Codable, and persist lightweight data with UserDefaults and the file system.
- Debug with breakpoints and LLDB, manage dependencies with Swift Package Manager, and write unit tests.
- Scope, build, test, and refactor a complete Swift command-line app, using AI assistance with human review.

SwiftUI and App Architecture **3.00** **90.00**

The SwiftUI and App Architecture course teaches students to build real, multi-screen iOS interfaces with SwiftUI. Students progress from layout and state fundamentals through navigation, animation, and adaptive design to the skills that separate a demo from a maintainable app: framework integration, testing, accessibility, and architecture.

Course Objectives:

- Compose adaptive layouts with stacks, grids, ScrollView, and GeometryReader that work across device sizes and orientations.
- Manage state correctly with @State, @Binding, @Observable, and Environment, maintaining a single source of truth.
- Implement multi-screen navigation with NavigationStack, tabs, sheets, and deep links.
- Design for the full data lifecycle (loading, error, and empty states) with resilient, recoverable UI.
- Find, evaluate, and integrate first- and third-party frameworks by working from their documentation.
- Test view models and UI flows (Swift Testing, XCUITest) and meet accessibility expectations (VoiceOver, Dynamic Type).
- Organize a growing codebase (MVVM, modularization) and defend architecture decisions in review.

Networking, Persistence and Cloud Backends **3.00** **90.00**

The Networking, Persistence and Cloud Backends course builds the data layer end to end. Students build the memory and concurrency foundations that networking depends on, then consume real APIs through a typed networking layer. They persist data locally with SwiftData, learn how relational databases are structured, and stand up cloud backends with Firebase. By the end, they can choose and justify the right data architecture for a given app.

Course Objectives:

- Manage memory and concurrency safely: ARC and retain cycles, async/await, actors, and main-thread rules.
- Build a reusable networking layer on URLSession that decodes complex JSON and handles errors, retries, and offline states.
- Design relational database structure: tables, keys, relationships, normalization, and joins.
- Persist and query structured data with SwiftData, and read legacy Core Data code.
- Implement authentication (tokens, OAuth, Sign in with Apple) and protect secrets with Keychain and build configurations.
- Build a Firebase/Firestore backend: data modeling, CRUD, auth, security rules, cloud functions, and push notifications.
- Compare local, BaaS, CloudKit, and custom-API options and justify a data-layer choice for a real project.

App Store, DevOps and App Marketing **2.00** **60.00**

The App Store, DevOps and App Marketing course covers everything between an app that works and an app that people are using. Students take a finished build through signing, TestFlight, and App Store review; automate testing and releases with CI/CD; monitor what happens after launch; and market the app with a brand, a website, and a social presence that feed into App Store optimization.

Course Objectives:

- Take an app from Xcode build through code signing, TestFlight beta testing, and App Store submission.
- Design and run a CI/CD pipeline (Xcode Cloud, fastlane, GitHub Actions) with automated test suites.
- Monitor release health with crash reporting, analytics, and alerting, and respond to regressions.
- Audit and optimize performance, app size, and accessibility before release.
- Create an app brand identity, build a marketing landing page, and plan a social media launch strategy.
- Optimize an App Store listing (ASO) and plan pricing, in-app purchases, and phased rollouts.

Capstone and Career Launch **2.00** **60.00**

The Capstone and Career Launch course has teams of two scope, build, and ship a complete iOS app through sprints, integration points, code reviews, and quality passes to a TestFlight release and public demo day. The final stretch of the course converts that work into career momentum: portfolio, interviews, and offers.

Course Objectives:

- Scope a buildable product, plan its architecture, and divide work across a two-person team.
- Ship working software in sprints, integrating early, triaging scope, and paying down technical debt.
- Run pre-release quality passes (testing, performance, accessibility, security) and distribute via TestFlight.
- Present and demo technical work to technical and non-technical audiences.
- Produce job-search assets: resume, portfolio, GitHub profile, and personal brand.
- Perform in technical and behavioral interviews, and evaluate and negotiate job offers.